

Name : Muhammad Owais
Reg No : FA24-BSE-054
Instructor : Sir Ehaz Mustafa
Lab #21/22:

BFS :

```
class queue{
    private final int size = 20;
    private int[] QueArray;
    private int front;
    private int rear;
    public queue(){
        QueArray = new int[size];
        front = 0;
        rear = -1;
    }
    public void enqueue(int j){
        if(rear == size -1){
            rear = -1;
        }
        QueArray[++rear] = j;
    }
    public int dequeue() {
        int temp = QueArray[front++];
        if (front == size) {
            front = 0;
        }
        return temp;
    }
    public boolean isEmpty() {
        return (rear + 1 == front || (front + size - 1 == rear));
    }
}

class vertex{
    public char label;
    public boolean wasVisited;
    public vertex(char label){
        this.label = label;
        wasVisited = false;
    }
}

class graph{
    private final int MAX_VERTS = 20;
    private vertex vertexList[];
    private int adjMat[][];
    private int nVerts;
    private queue theQueue;

    private int nVerts;
    private queue theQueue;
    public graph(){
        vertexList = new vertex[MAX_VERTS];
        adjMat = new int[MAX_VERTS][MAX_VERTS];
        nVerts = 0;
        for (int j = 0; j < MAX_VERTS; j++) {
            for (int k = 0; k < MAX_VERTS; k++) {
                adjMat[j][k] = 0;
            }
        }
        theQueue = new queue();
    }
    public void addVertex(char lab) {
        vertexList[nVerts++] = new vertex(lab);
    }
    public void addEdge(int start, int end) {
        adjMat[start][end] = 1;
        adjMat[end][start] = 1;
    }
    public void displayVertex(int v) {
        System.out.print(vertexList[v].label);
    }
    public void bfs() {
        vertexList[0].wasVisited = true;
        displayVertex(v: 0);
        theQueue.enqueue(j: 0);
        int v2;
        while (!theQueue.isEmpty()) { // until queue empty,
            int v1 = theQueue.dequeue();
            while ((v2 = getAdjUnvisitedVertex(v1)) != -1) {
                vertexList[v2].wasVisited = true;
                displayVertex(v2);
                theQueue.enqueue(v2);
            }
        }
        for (int j = 0; j < nVerts; j++) {
            vertexList[j].wasVisited = false;
        }
    }
}
```

```
80     public int getAdjUnvisitedVertex(int v) {
81         for (int j = 0; j < nVerts; j++) {
82             if (adjMat[v][j] == 1 && vertexList[j].wasVisited == false) {
83                 return j;
84             }
85         }
86         return -1;
87     }
88 }
89 public class BFS {
90     Run | Debug | Run main | Debug main
91     public static void main(String[] args) {
92         graph theGraph = new graph ();
93         theGraph.addVertex(lab: 'A');
94         theGraph.addVertex(lab: 'B');
95         theGraph.addVertex(lab: 'C');
96         theGraph.addVertex(lab: 'D');
97         theGraph.addVertex(lab: 'E');
98         theGraph.addEdge(start: 0, end: 1);
99         theGraph.addEdge(start: 1, end: 2);
100         theGraph.addEdge(start: 0, end: 3);
101         theGraph.addEdge(start: 3, end: 4);
102         System.out.print("Visits: ");
103         theGraph.bfs();
104         System.out.println();
105     }
```

OUTPUT :

```
● owais-hashim@owais-  
is-hashim/.config/C  
Visits: ABDCE
```

DFS :

```
1 import java.util.Stack;
2 class Vertex {
3     public char label;
4     public boolean wasVisited;
5     public Vertex(char lab){
6         label = lab;
7         wasVisited = false;
8     }
9 }
10 class Graph {
11     private final int MAX_VERTS = 20;
12     private Vertex vertexList[];
13     private int adjMat[][];
14     private int nVerts;
15     private Stack theStack;
16     public Graph(){
17         vertexList = new Vertex[MAX_VERTS];
18         adjMat = new int[MAX_VERTS][MAX_VERTS];
19         nVerts = 0;
20         for (int y = 0; y < MAX_VERTS; y++){
21             for (int x = 0; x < MAX_VERTS; x++){
22                 adjMat[x][y] = 0;
23             }
24         }
25         theStack = new Stack();
26     } // end constructor
27     public void addVertex(char lab) {
28         vertexList[nVerts++] = new Vertex(lab);
29     }
30     public void addEdge(int start, int end) {
31         adjMat[start][end] = 1;
32         adjMat[end][start] = 1;
33     }
34     public void displayVertex(int v) {
35         System.out.print(vertexList[v].label);
36     }
37     public void dfs(){
38         vertexList[0].wasVisited = true;
39         displayVertex(v: 0);
40         theStack.push(0);
41         while (!theStack.isEmpty()){
42             int v = getAdjUnvisitedVertex((int)theStack.peek());
43             if (v == -1){
44                 theStack.pop();
45             } else{
46                 vertexList[v].wasVisited = true;
47                 displayVertex(v);
48                 theStack.push(v);
49             }
50         }
51     }
52     public int getAdjUnvisitedVertex(int v){
53         for (int i = 0; i < nVerts; i++){
54             if(adjMat[v][i] == 1 && !vertexList[i].wasVisited){
55                 return i;
56             }
57         }
58         return -1;
59     }
60 }
61
62 public class DFS {
63     Run | Debug | Run main | Debug main
64     public static void main(String[] args) {
65         Graph theGraph = new Graph();
66         theGraph.addVertex(lab: 'A');
67         theGraph.addVertex(lab: 'B');
68         theGraph.addVertex(lab: 'C');
69         theGraph.addVertex(lab: 'D');
70         theGraph.addVertex(lab: 'E');
71         theGraph.addEdge(start: 0, end: 1);
72         theGraph.addEdge(start: 1, end: 2);
73         theGraph.addEdge(start: 0, end: 3);
74         theGraph.addEdge(start: 3, end: 4);
75         System.out.print("Visits: ");
76         theGraph.dfs();
77         System.out.println();
78     }
79 }
```

OUTPUT :

```
● owais-hashim@owais-  
etailsInExceptionMe  
Visits: ABCDE
```